1️⃣

Buổi 1: Microservices Mindset + Minimal API + CQRS với MediatR

Giáo án Buổi 1: Microservices Mindset + Minimal API + CQRS với MediatR

0. Mục tiêu buổi học

Sau buổi này, bạn cần làm được 5 việc:

1. Hiểu Microservices là gì và vì sao không nên tách service bừa bãi.
2. Hiểu Monolith, Coupling, Cohesion, Command, Query, Handler.
3. Tạo được một API đơn giản bằng Minimal API.
4. Refactor API sang CQRS cơ bản với MediatR.
5. Giải thích được request đi từ API endpoint đến Handler như thế nào.

> Kết quả cuối buổi: một `ProductService` có API tạo, xem, sửa, xóa sản phẩm; code được tách thành `Commands`, `Queries`, `Handlers`.

---

1. Kiến thức nền cần hiểu trước

1.1. Monolith là gì?

Monolith là kiểu ứng dụng mà nhiều chức năng lớn nằm chung trong một project/app.

Ví dụ một app e-commerce có:

- Product
- Order

- Payment
- User
- Shipping

Tất cả nằm chung trong một ứng dụng duy nhất.

Ưu điểm:

- Dễ bắt đầu.
- Dễ debug khi hệ thống nhỏ.
- Deploy đơn giản lúc ban đầu.

Nhược điểm khi hệ thống lớn:

- Một thay đổi nhỏ có thể phải deploy lại cả hệ thống.
- Scale phải scale nguyên app, không scale riêng từng phần được.
- Code dễ rối nếu team đông và domain lớn.

1.2. Microservices là gì?

Microservices là kiến trúc chia hệ thống thành nhiều service nhỏ, mỗi service phụ trách một năng lực nghiệp vụ riêng.

Ví dụ:

```
E-commerce System
├── Product Service
├── Basket Service
├── Order Service
├── Payment Service
└── Shipping Service
```

Mỗi service có thể:

- code riêng
- database riêng
- deploy riêng
- scale riêng

- team riêng chịu trách nhiệm

Nhưng Microservices cũng phức tạp hơn Monolith vì phải xử lý:

- giao tiếp giữa các service
- lỗi mạng
- dữ liệu phân tán
- logging/tracing
- deployment nhiều thành phần

> Ghi nhớ: Microservices không phải cứ tách càng nhỏ càng tốt. Tách đúng ranh giới nghiệp vụ mới quan trọng.

1.3. Coupling là gì?

Coupling là mức độ phụ thuộc giữa các thành phần.

Coupling cao:

```
A đổi → B dê˜lôĩ
B đổi → C dê˜lôĩ
```

Coupling thấp:

```
A đổi ít ảnh hưởng B
B có thểthay đổi độc lập hơn
```

Trong microservices, mục tiêu là giảm coupling để các service độc lập hơn.

Ví dụ coupling cao:

```
public class OrderService
{
    private readonly PaymentService _paymentService = new Pay
mentService();
}
```

Ví dụ coupling thấp hơn:

```
public class OrderService
{
    private readonly IPaymentService _paymentService;

    public OrderService(IPaymentService paymentService)
    {
        _paymentService = paymentService;
    }
}
```

1.4. Cohesion là gì?

Cohesion là mức độ các phần bên trong một module/service có liên quan chặt chẽ với nhau.

High Cohesion là tốt.

Ví dụ `ProductService` nên tập trung vào:

- tạo sản phẩm
- sửa sản phẩm
- lấy danh sách sản phẩm
- xóa sản phẩm

Không nên nhét thêm:

- thanh toán
- gửi email
- xử lý shipping

> Senior mindset: low coupling, high cohesion.

---

2. CQRS là gì?

CQRS là viết tắt của:

```
Command Query Responsibility Segregation
```

Hiểu đơn giản:

- Command: thao tác làm thay đổi dữ liệu.
- Query: thao tác chỉ đọc dữ liệu.

Ví dụ:

```
CreateProductCommand  → tạo sản phẩm       → ghi dữ liệu
UpdateProductCommand  → sửa sản phẩm       → ghi dữ liệu
DeleteProductCommand  → xóa sản phẩm       → ghi dữ liệu
GetProductsQuery      → lấy danh sách      → đọc dữ liệu
GetProductByIdQuery   → lấy 1 sản phẩm     → đọc dữ liệu
```

2.1. Vì sao CQRS giúp code dễ maintain?

Vì nó tách rõ 2 luồng:

```
API Endpoint
    ↓
Command / Query
    ↓
Handler
    ↓
Business/Data logic
```

Nếu không dùng CQRS, controller hoặc endpoint rất dễ bị nhồi nhiều logic:

- validate
- xử lý nghiệp vụ
- gọi database
- mapping dữ liệu
- trả response

Dùng CQRS giúp mỗi file có trách nhiệm rõ hơn.

2.2. MediatR là gì?

MediatR là thư viện giúp gửi Command/Query tới đúng Handler.

Thay vì endpoint gọi trực tiếp handler:

```
handler.Handle(command);
```

Ta dùng:

```
await mediator.Send(command);
```

MediatR sẽ tự tìm handler tương ứng.

Ví dụ:

```
mediator.Send(CreateProductCommand)
        ↓
CreateProductCommandHandler.Handle()
```

---

3. Chuẩn bị môi trường

Cần có:

- .NET SDK
- Rider / Visual Studio / VS Code
- Postman / Bruno / Thunder Client hoặc dùng file `.http`

Kiểm tra .NET:

```
dotnet --version
```

---

4. Phần thực hành 1: Tạo ProductService bằng Minimal API

4.1. Tạo project

```
dotnet new web -n ProductService
cd ProductService
```

Chạy thử:

```
dotnet run
```

4.2. Tạo model Product

Tạo file `Product.cs`:

```
namespace ProductService;

public class Product
{
    public string Id { get; set; } = Guid.NewGuid().ToString
();
    public string Name { get; set; } = string.Empty;
}
```

Giải thích:

- `Id`: định danh sản phẩm.
- `Name`: tên sản phẩm.
- `Guid.NewGuid().ToString()`: tạo Id tự động.

4.3. Viết API CRUD cơ bản chưa dùng CQRS

Mở `Program.cs` và sửa:

```
using ProductService;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
var products = new List<Product>();

app.MapGet("/products", () =>
{
    return Results.Ok(products);
});

app.MapGet("/products/{id}", (string id) =>
{
    var product = products.FirstOrDefault(p => p.Id == id);

    if (product is null)
        return Results.NotFound();

    return Results.Ok(product);
});

app.MapPost("/products", (CreateProductRequest request) =>
{
    if (string.IsNullOrWhiteSpace(request.Name))
        return Results.BadRequest("Product name is require
d.");

    var product = new Product
    {
        Name = request.Name
    };

    products.Add(product);

    return Results.Created($"/products/{product.Id}", produc
t);
});

app.MapPut("/products/{id}", (string id, UpdateProductRequest
request) =>
```

```
{
    var product = products.FirstOrDefault(p => p.Id == id);

    if (product is null)
        return Results.NotFound();

    if (string.IsNullOrWhiteSpace(request.Name))
        return Results.BadRequest("Product name is require
d.");

    product.Name = request.Name;

    return Results.Ok(product);
});

app.MapDelete("/products/{id}", (string id) =>
{
    var product = products.FirstOrDefault(p => p.Id == id);

    if (product is null)
        return Results.NotFound();

    products.Remove(product);

    return Results.NoContent();
});

app.Run();

public record CreateProductRequest(string Name);
public record UpdateProductRequest(string Name);
```

4.4. Tạo file test API

Tạo file `ProductService.http`:

```
@host = <http://localhost:5000>

### Get all products
GET {{host}}/products

### Create product
POST {{host}}/products
Content-Type: application/json

{
  "name": "iPhone 15"
}

### Get product by id
GET {{host}}/products/{{productId}}

### Update product
PUT {{host}}/products/{{productId}}
Content-Type: application/json

{
  "name": "iPhone 15 Pro"
}

### Delete product
DELETE {{host}}/products/{{productId}}
```

Nếu port khác, chỉnh lại `@host` theo console khi chạy `dotnet run`.

---

5. Vấn đề của code hiện tại

Code hiện tại chạy được, nhưng có vấn đề:

```
Program.cs đang làm quá nhiều việc:
- định nghĩa endpoint
- validate request
- tạo product
- tìm product
- update product
- delete product
- giữ data trong List
```

Khi nghiệp vụ lớn hơn, `Program.cs` sẽ phình to.

Vì vậy ta refactor sang CQRS + MediatR.

6. Phần thực hành 2: Refactor sang CQRS + MediatR

6.1. Cài MediatR

```
dotnet add package MediatR
```

6.2. Tạo cấu trúc thư mục

Tạo các folder:

```
ProductService/
├── Commands/
├── Queries/
├── Handlers/
├── Product.cs
├── ProductStore.cs
└── Program.cs
```

6.3. Tạo ProductStore

Tạo file `ProductStore.cs`:

```
namespace ProductService;

public class ProductStore
{
    public List<Product> Products { get; } = new();
}
```

Giải thích:

- Đây là nơi lưu data tạm thời trong RAM.
- Buổi sau sẽ thay bằng database thật.
- Dùng `ProductStore` để nhiều handler dùng chung một danh sách sản phẩm.

7. Tạo Commands

7.1. CreateProductCommand

Tạo file `Commands/CreateProductCommand.cs`:

```
using MediatR;

namespace ProductService.Commands;

public record CreateProductCommand(string Name) : IRequest<Pr
oduct>;
```

Giải thích:

- Đây là command tạo sản phẩm.
- Vì tạo xong trả về `Product`, nên dùng `IRequest<Product>`.

7.2. UpdateProductCommand

Tạo file `Commands/UpdateProductCommand.cs`:

```
using MediatR;

namespace ProductService.Commands;

public record UpdateProductCommand(string Id, string Name) :
IRequest<Product?>;
```

Giải thích:

- Update có thể không tìm thấy product.
- Vì vậy trả về `Product?`.

7.3. DeleteProductCommand

Tạo file `Commands/DeleteProductCommand.cs`:

```
using MediatR;

namespace ProductService.Commands;

public record DeleteProductCommand(string Id) : IRequest<bool
>;
```

Giải thích:

- Trả về `true` nếu xóa thành công.
- Trả về `false` nếu không tìm thấy.

---

8. Tạo Queries

8.1. GetProductsQuery

Tạo file `Queries/GetProductsQuery.cs`:

```
using MediatR;

namespace ProductService.Queries;

public record GetProductsQuery : IRequest<List<Product>>;
```

8.2. GetProductByIdQuery

Tạo file `Queries/GetProductByIdQuery.cs`:

```
using MediatR;

namespace ProductService.Queries;

public record GetProductByIdQuery(string Id) : IRequest<Produ
ct?>;
```

---

9. Tạo Handlers

9.1. CreateProductHandler

Tạo file `Handlers/CreateProductHandler.cs`:

```
using MediatR;
using ProductService.Commands;

namespace ProductService.Handlers;

public class CreateProductHandler : IRequestHandler<CreatePro
ductCommand, Product>
{
    private readonly ProductStore _store;

    public CreateProductHandler(ProductStore store)
    {
        _store = store;
    }

    public Task<Product> Handle(CreateProductCommand request,
CancellationToken cancellationToken)
    {
        var product = new Product
        {
            Name = request.Name
        };

        _store.Products.Add(product);

        return Task.FromResult(product);
    }
}
```

9.2. GetProductsHandler

Tạo file `Handlers/GetProductsHandler.cs`:

```
using MediatR;
using ProductService.Queries;

namespace ProductService.Handlers;

public class GetProductsHandler : IRequestHandler<GetProducts
Query, List<Product>>
{
    private readonly ProductStore _store;

    public GetProductsHandler(ProductStore store)
    {
        _store = store;
    }

    public Task<List<Product>> Handle(GetProductsQuery reques
t, CancellationToken cancellationToken)
    {
        return Task.FromResult(_store.Products);
    }
}
```

9.3. GetProductByIdHandler

Tạo file `Handlers/GetProductByIdHandler.cs`:

```
using MediatR;
using ProductService.Queries;

namespace ProductService.Handlers;

public class GetProductByIdHandler : IRequestHandler<GetProdu
```

```
ctByIdQuery, Product?>
{
    private readonly ProductStore _store;

    public GetProductByIdHandler(ProductStore store)
    {
        _store = store;
    }

    public Task<Product?> Handle(GetProductByIdQuery request,
CancellationToken cancellationToken)
    {
        var product = _store.Products.FirstOrDefault(p => p.I
d == request.Id);

        return Task.FromResult(product);
    }
}
```

9.4. UpdateProductHandler

Tạo file `Handlers/UpdateProductHandler.cs`:

```
using MediatR;
using ProductService.Commands;

namespace ProductService.Handlers;

public class UpdateProductHandler : IRequestHandler<UpdatePro
ductCommand, Product?>
{
    private readonly ProductStore _store;

    public UpdateProductHandler(ProductStore store)
    {
```

```
        _store = store;
    }

    public Task<Product?> Handle(UpdateProductCommand reques
t, CancellationToken cancellationToken)
    {
        var product = _store.Products.FirstOrDefault(p => p.I
d == request.Id);

        if (product is null)
            return Task.FromResult<Product?>(null);

        product.Name = request.Name;

        return Task.FromResult<Product?>(product);
    }
}
```

9.5. DeleteProductHandler

Tạo file `Handlers/DeleteProductHandler.cs`:

```
using MediatR;
using ProductService.Commands;

namespace ProductService.Handlers;

public class DeleteProductHandler : IRequestHandler<DeletePro
ductCommand, bool>
{
    private readonly ProductStore _store;

    public DeleteProductHandler(ProductStore store)
    {
        _store = store;
```

```
    }

    public Task<bool> Handle(DeleteProductCommand request, Ca
ncellationToken cancellationToken)
    {
        var product = _store.Products.FirstOrDefault(p => p.I
d == request.Id);

        if (product is null)
            return Task.FromResult(false);

        _store.Products.Remove(product);

        return Task.FromResult(true);
    }
}
```

---

10. Program.cs sau khi áp dụng MediatR

Sửa `Program.cs`:

```
using MediatR;
using ProductService;
using ProductService.Commands;
using ProductService.Queries;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<ProductStore>();

builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssembly(typeof(Program).Assembl
y));

var app = builder.Build();
```

```
app.MapGet("/products", async (IMediator mediator) =>
{
    var products = await mediator.Send(new GetProductsQuery
());
    return Results.Ok(products);
});

app.MapGet("/products/{id}", async (string id, IMediator medi
ator) =>
{
    var product = await mediator.Send(new GetProductByIdQuery
(id));

    if (product is null)
        return Results.NotFound();

    return Results.Ok(product);
});

app.MapPost("/products", async (CreateProductRequest request,
IMediator mediator) =>
{
    if (string.IsNullOrWhiteSpace(request.Name))
        return Results.BadRequest("Product name is require
d.");

    var product = await mediator.Send(new CreateProductComman
d(request.Name));

    return Results.Created($"/products/{product.Id}", produc
t);
});

app.MapPut("/products/{id}", async (string id, UpdateProductR
equest request, IMediator mediator) =>
```

```
{
    if (string.IsNullOrWhiteSpace(request.Name))
        return Results.BadRequest("Product name is require
d.");

    var product = await mediator.Send(new UpdateProductComman
d(id, request.Name));

    if (product is null)
        return Results.NotFound();

    return Results.Ok(product);
});

app.MapDelete("/products/{id}", async (string id, IMediator m
ediator) =>
{
    var deleted = await mediator.Send(new DeleteProductComman
d(id));

    if (!deleted)
        return Results.NotFound();

    return Results.NoContent();
});

app.Run();

public record CreateProductRequest(string Name);
public record UpdateProductRequest(string Name);
```

---

11. Luồng chạy cần hiểu

11.1. Tạo sản phẩm

```
POST /products
    ↓
Minimal API endpoint
    ↓
mediator.Send(CreateProductCommand)
    ↓
CreateProductHandler.Handle()
    ↓
ProductStore.Products.Add(product)
    ↓
return 201 Created
```

11.2. Lấy danh sách sản phẩm

```
GET /products
    ↓
Minimal API endpoint
    ↓
mediator.Send(GetProductsQuery)
    ↓
GetProductsHandler.Handle()
    ↓
return products
```

---

12. Bài tập chính

Bài 1: Tạo API cơ bản chưa dùng CQRS

Yêu cầu:

- `GET /products`
- `GET /products/{id}`
- `POST /products`
- `PUT /products/{id}`
- `DELETE /products/{id}`

Điều kiện đạt:

- tạo được product

- lấy được danh sách
- lấy được 1 product theo id
- sửa được product
- xóa được product
- response đúng `Ok`, `Created`, `NotFound`, `BadRequest`, `NoContent`

Bài 2: Refactor sang CQRS + MediatR

Tạo đủ các file:

```
Commands/
├── CreateProductCommand.cs
├── UpdateProductCommand.cs
└── DeleteProductCommand.cs

Queries/
├── GetProductsQuery.cs
└── GetProductByIdQuery.cs

Handlers/
├── CreateProductHandler.cs
├── UpdateProductHandler.cs
├── DeleteProductHandler.cs
├── GetProductsHandler.cs
└── GetProductByIdHandler.cs
```

Bài 3: Giải thích flow

Trả lời bằng lời của bạn:

1. Command khác Query ở đâu?
2. Handler dùng để làm gì?
3. MediatR đứng ở giữa để làm gì?
4. Request `POST /products` đi qua những bước nào?
5. Vì sao CQRS giúp code dễ maintain hơn?

Bài 4: Bài nâng cấp nhỏ

Thêm API:

```
GET /products/search?keyword=abc
```

Yêu cầu:

- Tạo `SearchProductsQuery`
- Tạo `SearchProductsHandler`
- Nếu `keyword` rỗng thì trả về toàn bộ sản phẩm
- Nếu có `keyword`, tìm theo `Name.Contains(keyword)`

Gợi ý command/query:

```
public record SearchProductsQuery(string? Keyword) : IRequest
<List<Product>>;
```

---

13. Đáp án bài nâng cấp Search

13.1. SearchProductsQuery.cs

```
using MediatR;

namespace ProductService.Queries;

public record SearchProductsQuery(string? Keyword) : IRequest
<List<Product>>;
```

13.2. SearchProductsHandler.cs

```
using MediatR;
using ProductService.Queries;

namespace ProductService.Handlers;
```

```
public class SearchProductsHandler : IRequestHandler<SearchPr
oductsQuery, List<Product>>
{
    private readonly ProductStore _store;

    public SearchProductsHandler(ProductStore store)
    {
        _store = store;
    }

    public Task<List<Product>> Handle(SearchProductsQuery req
uest, CancellationToken cancellationToken)
    {
        if (string.IsNullOrWhiteSpace(request.Keyword))
            return Task.FromResult(_store.Products);

        var result = _store.Products
            .Where(p => p.Name.Contains(request.Keyword, Stri
ngComparison.OrdinalIgnoreCase))
            .ToList();

        return Task.FromResult(result);
    }
}
```

13.3. Endpoint trong Program.cs

```
app.MapGet("/products/search", async (string? keyword, IMedia
tor mediator) =>
{
    var products = await mediator.Send(new SearchProductsQuer
y(keyword));
```

```
    return Results.Ok(products);
});
```

Nhớ thêm namespace:

```
using ProductService.Queries;
```

---

14. Checklist tự chấm

Bạn đạt buổi 1 nếu tick được các mục này:

```
[ ] Tạo được Minimal API project
[ ] Viết được CRUD Product cơ bản
[ ] Biết trả HTTP response phù hợp
[ ] Tách được Commands
[ ] Tách được Queries
[ ] Tạo được Handler tương ứng
[ ] Đăng ký MediatR trong Program.cs
[ ] Endpoint gọi qua IMediator
[ ] Giải thích được Command vs Query
[ ] Giải thích được request flow API → Mediator → Handler
```

---

15. Những lỗi hay gặp

15.1. Quên đăng ký MediatR

Lỗi thường gặp:

```
Unable to resolve service for type 'MediatR.IMediator'
```

Cách sửa:

```
builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssembly(typeof(Program).Assembl
y));
```

15.2. Handler không được tìm thấy

Nguyên nhân thường gặp:

- Handler không implement đúng `IRequestHandler<TRequest, TResponse>`
- Command/Query khai báo response type không khớp
- namespace sai

Ví dụ đúng:

```
public record GetProductsQuery : IRequest<List<Product>>;

public class GetProductsHandler : IRequestHandler<GetProducts
Query, List<Product>>
{
    // ...
}
```

15.3. Dùng List trong RAM bị mất dữ liệu

Đây không phải lỗi ở buổi 1.

Buổi 1 dùng `List` để học flow.

Buổi 2 sẽ thay bằng database.

---

16. Từ vựng cần nhớ

Từ	Nghĩa dễ hiểu
Monolith	Ứng dụng nguyên khối
Microservices	Hệ thống gồm nhiều service nhỏ
Coupling	Mức độ phụ thuộc giữa các phần
Loose Coupling	Phụ thuộc thấp, dễ thay đổi
Cohesion	Độ tập trung trách nhiệm bên trong module
Command	Yêu cầu ghi/thay đổi dữ liệu
Query	Yêu cầu đọc dữ liệu

Từ	Nghĩa dễ hiểu
Handler	Class xử lý command/query
MediatR	Thư viện điều phối request tới handler
Minimal API	Cách viết API gọn trong ASP.NET Core

17. Câu hỏi ôn tập

1. Vì sao không nên nhét toàn bộ logic vào `Program.cs`?
2. `CreateProductCommand` khác `GetProductsQuery` ở điểm nào?
3. `IRequest<Product>` nghĩa là gì?
4. Vì sao `DeleteProductCommand` có thể trả về `bool`?
5. Khi nào endpoint nên trả `NotFound`?
6. Khi nào endpoint nên trả `BadRequest`?
7. ProductStore hiện tại có nhược điểm gì?
8. Nếu sau này dùng database, Handler sẽ thay đổi như thế nào?

18. Format gửi bài để được review

Khi làm xong, gửi theo mẫu:

```
Buổi 1 - Bài nộp

1. Câú trúc thư mục:
(paste ảnh hoặc text)

2. Code chính:
- Program.cs
- Product.cs
- ProductStore.cs
- Commands
- Queries
- Handlers

3. Cách hiểủ của tôi:
- Command là ...
- Query là ...
- MediatR dùng để...
```

```
- Flow POST /products là ...

4. Chỗ chưa hiểu:
- ...
```

---

19. Điều kiện mở khóa Buổi 2

Bạn sẽ sang Buổi 2 khi làm được:

- CRUD Product chạy được
- Refactor qua MediatR chạy được
- Giải thích đúng flow request

Buổi 2 sẽ học:

```
Database + Repository + Dapper/EF Core mindset + chuyển ProductStore sang DB thật
```

✅ Chấm nhanh 17. Câu hỏi ôn tập

```
Điểm: 8.5/10
```

Bro hiểu đúng phần lõi:

- `Program.cs` không nên ôm logic
- Command khác Query
- `IRequest<Product>` là response type
- `ProductStore` in-memory bị mất data
- Sau này Handler nên làm việc qua lớp truy cập dữ liệu

Góp ý từng câu

1. Vì sao không nên nhét toàn bộ logic vào `Program.cs`?

Bro trả lời đúng hướng, nhưng nên bổ sung thêm: `Program.cs` là **composition root / entry point**, nhiệm vụ chính là cấu hình app, middleware, DI, route. Không nên biến nó thành nơi chứa business logic.

Câu chuẩn hơn:

```
Không nên nhét toàn bộ logic vào Program.cs vì Program.cs chỉ
nên là nơi câú hình app và khai báo endpoint. Nêú nhôì busine
ss logic vào đây, code sẽ khó đọc, khó test, khó mở rộng, end
point bị phụ thuộc trực tiêp vào logic xửlý và sau này rât k
hó maintain.
```

---

2. `CreateProductCommand` khác `GetProductsQuery` ở điểm nào?

Bro trả lời đúng, nhưng mới nói về Command. Nên nói rõ cả 2 phía:

```
CreateProductCommand là command, dùng cho thao tác ghi/thay đôỉ dữ liệu, ví dụ tạo produc
t. GetProductsQuery là query, dùng cho thao tác đọc dữ liệu, không làm thay đôỉ trạng thá
i hệ thôńg.
```

---

3. `IRequest<Product>` nghĩa là gì?

Câu này bro trả lời **chuẩn**.

Chỉnh câu cho gọn hơn:

```
IRequest<Product> nghĩa là request này khi được MediatR xửlý sẽ trả vềkêt quả kiêủ Prod
uct.
```

Ví dụ:

```
publicrecordCreateProductCommand(stringName) :IRequest<Product>;
```

Nghĩa là:

```
Send command này → Handler xử lý → trả về Product
```

---

4. Vì sao `DeleteProductCommand` có thể trả về `bool`?

Bro hiểu đúng. Bổ sung thêm chút:

```
DeleteProductCommand có thể trả về bool vì sau khi xóa không cần trả về object Product nữ
a. Chỉ cần biết xóa thành công hay thất bại. true = tìm thấy và xóa được, false = không t
ìm thấy product để xóa.
```

---

5. Khi nào endpoint nên trả `NotFound`?

Câu này của bro đúng nhưng hơi lẫn 2 ý.

Có 2 loại:

1. Route không tồn tại

Ví dụ gọi `/abc` mà app không map endpoint nào. Cái này framework tự trả 404.

2. Resource không tồn tại

Ví dụ endpoint có tồn tại:

```
GET /products/{id}
```

nhưng không có product với id đó. Đây là chỗ mình chủ động return:

```
returnResults.NotFound();
```

Câu chuẩn hơn:

```
Endpoint nên trả NotFound khi route hợp lệ nhưng tài nguyên cần tìm không tồn tại, ví dụ
GET /products/{id} nhưng không tìm thấy product có id đó.
```

---

6. Khi nào endpoint nên trả `BadRequest`?

Bro trả lời tốt.

Câu chuẩn hơn:

```
BadRequest dùng khi request gửi lên sai hoặc thiếu dữ liệu, ví dụ Name rỗng, Price âm, fo
rmat không hợp lệ, hoặc vi phạm rule validate phía server.
```

---

7. ProductStore hiện tại có nhược điểm gì?

Bro trả lời đúng.

Bổ sung thêm mấy ý production:

```
ProductStore chỉ lưu in-memory nên mất dữ liệu khi app restart. Ngoài ra nó không phù hợp
khi chạy nhiều instance, không có transaction, không query tốt khi dữ liệu lớn và làm Han
dler phụ thuộc trực tiếp vào cách lưu dữ liệu hiện tại.
```

Câu này sẽ rất liên quan Buổi 2.

---

8. Nếu sau này dùng database, Handler sẽ thay đổi như thế nào?

Bro trả lời đúng hướng nhưng cần chỉnh một chút. Không nên nói Handler làm việc với "đối tượng cụ thể ở tầng DB", vì mục tiêu của Buổi 2 là **Handler không phụ thuộc trực tiếp DB cụ thể**.

Câu chuẩn hơn:

```
Khi dùng database, Handler không nên làm việc trực tiếp với database hay SQL. Handler nên
gọi qua một abstraction như IProductRepository. Repository sẽ chịu trách nhiệm làm việc v
ới database thật. Như vậy Handler chỉ tập trung xử lý use case, còn cách lưu dữ liệu nằm
ở Repository.
```

Flow chuẩn Buổi 2:

```
API → MediatR → Handler → IProductRepository → ProductRepository → Database
```

✅ Kết luận

Bro pass phần ôn tập Buổi 1 rồi. Câu cần sửa mindset nhất là **câu 8**:

> Handler không nên phụ thuộc trực tiếp vào DB cụ thể, mà nên phụ thuộc vào interface như `IProductRepository`.

Đây chính là cầu nối sang Buổi 2.